

SpaceLoop LoopBot Architecture

A Multi-Tier RAG-Powered Marketplace Orchestrator Combining Semantic Retrieval, Real-Time Availability DB, and Deterministic Pricing

Document Type:	System Architectural Blueprint & RAG Specification	Target Platform:	SpaceLoop P2P Marketplace (Web / Mobile)
AI Architecture:	Dynamic Intent Router + 5 Subsystem Engines	Verification Status:	100% Audit Passed (8/8 Query Paths, 14/14 Suites)
Retrieval Pipeline:	Vector Embeddings (Cosine Sim) + BM25 Token Hybrid	Fallback Protocol:	Gemini 1.5 → Groq 70B → Deterministic Rules

1. Architectural Paradigm: The Orchestrator vs. Naive LLM

Conventional AI chatbots in marketplace applications suffer from significant limitations: passing every user query through a uniform Retrieval-Augmented Generation (RAG) vector database causes severe hallucination risks on numerical quantities (such as pricing, deposit calculation, and seat counts), latency spikes, and failure to validate real-time database state (such as conflicting booking overlaps). LoopBot implements an **Orchestrator Architecture** that treats RAG as only one specialized tool among multiple deterministic platform systems.

Core Principle: 'Don't send every query through RAG.' When a user asks for available time slots or total rental cost, querying static vector embeddings produces outdated or invented numbers. Instead, LoopBot dynamically decomposes user intent and routes execution to the single source of truth.

Table 1.1: LoopBot Subsystem Routing Matrix (Query Intent Mapping)

User Query Example	Subsystem Routed	Underlying Source of Truth	Architectural Rationale
"What amenities does this space have?"	RAG Retrieval	Space technical specs & amenities vector chunks	Retrieves semantic descriptions of hardware, power outlets, acoustic levels, and monitors.
"Can I bring food?"	RAG Retrieval	Listing house rules & food/beverage policies	Semantic extraction of snack/coffee permissions vs prohibited hot catering/greasy meals.
"Is this good for a team meeting?"	RAG + Listing Data	Structured DB capacity + RAG equipment specs	Combines hard max_capacity and sqft checks with acoustic dB isolation & display kit.
"Find spaces for 8 people"	Marketplace Search	Space SQL model + capacity/budget filters	Hard filtering on max_capacity >= 8, category match, location proximity, and ranking.
"What's available tomorrow?"	Availability DB	Booking table collision detection engine	Validates real-time session overlaps (start < end_req and end > start_req) for target date.
"How much for 4 hours?"	Pricing DB / Calculation	Deterministic rate engine + UPI escrow formula	Exact line-item calculation: Rent (Rate × 4h) + 5% platform fee + █100 refundable escrow.
"Book this space"	Booking System	Reservation workflow & Digital Pass router	Direct /space/{id} reservation payload, Section 52 micro-lease agreement & 4-digit arrival PIN.
"Compare these spaces"	DB + RAG	Dual-source comparison matrix	Side-by-side structured comparison (Rate, Capacity, OTI trust score) + semantic rules & reviews.

2. Hierarchical System Tree Architecture

LoopBot functions as an asynchronous pipeline with five distinct operational tiers. Incoming messages are intercepted by the intent detector, which invokes the relevant subsystem, prepares ground truth facts, and synthesizes the final explainable response.

Loop Bot Pipeline | +-- [Tier 1] Intent Detection & Entity Extractor | +-- Target Space Resolution (Context space_id or Lexical Match) | +-- Entity Extraction (Hours, Capacity/Pax, Date, Budget Ceiling) | +-- Intent Classifier (RAG, Suitability, Search, Availability, Pricing, Booking, Compare) | +-- [Tier 2A] Marketplace Search Engine | +-- Deterministic Hard Filters (min_capacity, max_price, category, location) | +-- Real-Time Availability Filter (Overlap check against active Booking rows) | +-- Pricing Summary (Standard 2h session estimation) | +-- [Tier 2B] Semantic RAG Retrieval Subsystem | +-- Listing Descriptions (Dimensions, lighting, acoustic dB, power access) | +-- Verified Amenities (Hardware equipment, fiber wifi, monitors, backup) | +-- House Rules & Policies (Food & beverage guidelines, smoking, quiet hours) | +-- Host Policies & Legal Framework (Section 52 revocable license, 50m geofence pass) | +-- Telemetry & Reviews (OTI trust index, on-time vacate rate, cleanliness score) | +-- [Tier 2C] Real-Time Availability & Pricing DB Engines | +-- Availability DB: Query active bookings for date window (08:00 AM - 09:00 PM) | +-- Pricing DB: $Total = (Rate \times Hours) + 5\% \text{ Fee} + 100 \text{ INR Refundable Escrow}$ | +-- [Tier 3] Recommendation & Composite Reranking | +-- Explainable Badges ("Top Pick", "Best Match", "Available Now") | +-- Multi-Criteria Fit Scoring (Capacity, Budget, Proximity, Noise level) | +-- [Tier 4] Response Generation & Plaintext Sanitization +-- Primary: Google Gemini 1.5 Pro / Flash Multi-Turn API +-- Secondary: Groq GPT-OSS 120B / 20B High-Throughput Engine +-- Deterministic Rule Fallback: 100% Zero-Downtime Local Heuristics

3. Subsystem Engineering Deep-Dive

3.1 Semantic RAG Retrieval Engine (backend/modules/ai/loopbot_orchestrator.py)

The RAG knowledge base indexes rich semantic chunks dynamically synthesized from production listing records and regulatory legal documentation:

- **Listing Specs:** Square footage, natural/ambient lighting ratings, acoustic dB noise floors (<35 dB for studios, <45 dB for focus rooms), and heavy-duty power circuit ratings.
- **House Rules & Food Policies:** Explicit demarcations between permitted dry snacks/sealed beverages and restricted hot catering or oily foods.
- **Section 52 Framework:** Explanations of revocable licenses under Section 52 of the Indian Easements Act, 1882, affirming zero tenancy claims.
- **Vector Search:** Generates dense embeddings using backend.modules.search.embedding.generate_embedding. Query relevance is calculated via Cosine Similarity blended with token overlap (70% vector + 30% keyword match) for rapid, explainable retrieval.

3.2 Real-Time Availability Collision Engine

To eliminate double-bookings and phantom availability, LoopBot's Availability DB queries the bookings table directly. A temporal collision is declared if and only if:

Booking.start_time < requested_end AND Booking.end_time > requested_start AND status IN ('confirmed', 'active'). LoopBot evaluates candidate spaces across standard operating hours (08:00 AM - 09:00 PM) and returns explicit free intervals for target dates.

3.3 Deterministic Pricing DB & UPI Escrow Protocol

SpaceLoop strictly avoids dynamic surge pricing or hidden convenience fees. Pricing calculations follow an immutable mathematical contract:

1. **Base Rent** = price_hourly x hours
2. **SpaceLoop Safety Fee** = Round(Base Rent x 0.05, 2)
3. **Refundable UPI Escrow Deposit** = 100.00 INR (held securely; refunded instantly via UPI penny-drop reverse upon on-time checkout)
4. **Total Upfront Payable** = Base Rent + Platform Fee + 100.00 INR
5. **Net Cost to Seeker** = Base Rent + Platform Fee

3.4 Side-by-Side Space Comparison (DB + RAG)

When evaluating options ('Compare these spaces'), LoopBot aggregates both structured database telemetry (rate, capacity, OTI host trust score, verified reviews) and qualitative RAG policies (food rules, acoustic isolation, studio lighting). It formats clear, side-by-side cards and provides targeted recommendations based on whether the seeker prioritizes focus work vs collaborative meetings.

4. Frontend Integration & Contextual Floating UI

LoopBot's frontend component (frontend/src/components/common/LoopBot.tsx) operates as a non-blocking, responsive floating assistant that coexists seamlessly with the underlying SpaceLoop application.

Feature Component	Engineering Implementation	User Experience Benefit
Draggable Window	Header handle with pointer event tracking (mouse & touch) and 8px viewport boundary clamping.	Users can reposition LoopBot anywhere without losing the window off-screen.
Corner Resizing	Angled dotted grip handle at bottom-right corner; bounded between 310x360px and 720x850px.	Allows compact glance mode or expanded view for lengthy space comparisons.
Non-Blocking Background	Removed full-screen modal overlays and black backdrops; pointer-events: auto only on widget.	Users can scroll listings, click buttons, and view space photos while actively chatting.
Active Route Context	Hooks into useLocation() to detect /space/:id; passes space_id to API.	LoopBot automatically knows which space the user is looking at without asking.
Dynamic Topic Chips	Context-aware chip array: switches to Amenities, Food Policy, Meeting Fit on space pages.	Enables one-click intent exploration without typing complex prompts.

5. Verification, Audit Suite & Production Readiness

All 8 specialized routing pathways and 14 end-to-end platform workflows have been tested and verified across HTTP endpoints:

Audit Checkpoint	Test Input / Endpoint	Status	Measured Output / Response
------------------	-----------------------	--------	----------------------------

Query 1: RAG Amenities	POST /api/ai/chat (space_id=4)	PASS	Returned ESD Work Mat, Weller Soldering Station, Fume Extractor, Power Sockets.
Query 2: RAG Food Policy	POST /api/ai/chat (space_id=4)	PASS	Verified snack & drink permissions; confirmed 100% smoke-free premises rule.
Query 3: Meeting Suitability	POST /api/ai/chat (space_id=4)	PASS	Combined 4-person capacity, 350 sqft layout, and noise floor into suitability report.
Query 4: Marketplace Search	POST /api/ai/chat ('Find spaces for 8 people')	PASS	Filtered 3 spaces with max_capacity >= 8, calculated pricing and availability badges.
Query 5: Availability DB	POST /api/ai/chat ('What's available tomorrow?')	PASS	Computed active session schedule; reported full day open (08:00 AM - 09:00 PM).
Query 6: Pricing Calculation	POST /api/ai/chat ('How much for 4 hours?')	PASS	Line items: Base 340 INR + Fee 17 INR + Escrow 100 INR = Total 457 INR (Net 357 INR).
Query 7: Booking System	POST /api/ai/chat ('Book this space')	PASS	Generated direct link /space/4 and 4-step reservation protocol with arrival PIN.
Query 8: Space Comparison	POST /api/ai/chat ('Compare these spaces')	PASS	Dual-space breakdown (Rate, Amenities, Noise, OTI Trust score, Recommendation).
Full Subsystem Audit	test_all_features_functional.py	14/14 PASS	100% operational across KYC, Booking, Micro-Lease, Geofence Check-In, Escrow.